Jetpack Compose Phases — Practice Class

Jetpack Compose — Practice Class

This practice class follows Lecture 5 (Jetpack Compose Phases: Composition, Layout and Drawing). Each question gives a short setup (Context) and what you need to answer or do with it (Task). THEORETICAL questions are answered in your own words. HANDS-ON questions are done in Android Studio, all in one project, `FramePhases` *(package* `uz.ttpu.framephases`*, Kotlin, Jetpack Compose, Material 3, Minimum SDK API 24), where you make Compose tell you what it does by writing to Logcat. Show one demo at a time from* `setContent { MaterialTheme { … } }`*. Plan for about two class sessions. Snippets leave out* `package` *and* `import` *lines: press Alt+Enter (Option+Return on macOS) and let Android Studio add them. In Logcat, type* `tag:Phases` *(or the tag named in the question) into the search field to see only your lines.*

Part 1 — The frame (Questions 1–3)

Question 1 — The three phases of a frame THEORETICAL

Context: Compose produces a frame in three phases. The lecture defines them as Composition (what UI to show), Layout (where to place it) and Drawing (how it renders). It calls their one-way order "unidirectional data flow" and names `BoxWithConstraints`, `LazyColumn` and `LazyRow` as notable exceptions.

Task: (a) Put the three phases in order and say in one sentence what each one outputs. (b) The Layout phase has two steps: name them. (c) Explain the exception: why does the composition of the children of a `LazyColumn` depend on its parent's layout phase? Give an example from a real screen. (d) Conceptually every phase runs for every frame, so why is Compose still fast? Name the mechanism that lets it skip work.

a) The order is composition , layout and then drawing. Composition : what we see (ui user interface) , layout : where we place ui output is size and the positions. Drawing : decides how it renders , ends up as pixels.

b) 2 steps in layout phase , measurements (size) and placement (positions)

c) Lazy column to compose the visible items. Depends on inheritance that takes from the parent , so firrst the layout phase (parent phase) For example a list od contacts or chat list , on a tall screen it will feet on 12 rows so 12 are composed and if its small screen that only can feet 8 8 will compose.boxwithconstrains is the same depens on layout phase.

d) Its fast cause it does not redo everthing on every frame , It tracks which state each part of the UI reads, and when a state changes it re-runs only the phases and restart scopes that read that state.smart recomposition.

Question 2 — Walk the layout algorithm THEORETICAL

Context: The layout phase traverses the UI tree with a three-step algorithm: measure children, decide own size, place children. Take this tree: a `Row` that contains an `Image` and a `Column`, and the `Column` contains two `Text`s. The leaves decide their own sizes (width × height, in pixels): Image 48×48, Text A 100×20, Text B 80×20. The `Row` places children left to right, the `Column` top to bottom.

Task: (a) Write the order of all measure and place events, the way the lecture does. (b) Give the final size of the `Column` and of the `Row`, and the (x, y) position of every node relative to the top-left corner of the `Row`. (c) How many times is each node visited, and why does the lecture stress this? What would happen to the traversal time if every node were measured twice at each level of the tree? (d) Which phase produced this tree, and what does each node in it contain?

a) measuree events: Row, Image, Column, Text A, Text B. Place events: Row, Image, Column, Text A, Text B. (Each parent measures its children, then decides its own size, then places its children.)

b) So in column children are one below the other so a 80 b 80 the max is (100,80) = 100 since they are below height is 20+20 = 40 column 100 40 for row since they are next to each oter well add the width i

48+ c 100 = 148 (width) height is i 48 + c48 (48,40) max = 48 row 148, 28 x increases to the right y increases to below .

c) So every node is visited once for measuring and once for placing , if evey node were measured twice the process would double and grow exponentialy . (o natition and time.)

d) The tree was produced by the Composition phase. Each node contains the information the layout phase needs: its modifiers, its measure and place logic (how to measure and place its children), and its children. After layout, each node also has its size and position.

Question 3 — State reads and restart scopes THEORETICAL

Context: When a snapshot state is read during a phase, Compose records what it was doing at that moment, so it can run the reader again when the value changes. A block of code that can be re-executed this way is a restart scope.

Task: (a) Snippets 1 and 2 read the same kind of state in two ways. Are they equivalent? At which moment does the read happen in each: when the state is created, or when it is used? (b) In snippet 3, `Text("A")` and `Text("B")` look the same on screen today. In which phase is `offsetX` read for `Text("A")` and in which for `Text("B")`? What happens in each case when `offsetX` changes? (c) In your own words: what is a restart scope, and which block is the restart scope for a read made directly in the body of a composable function?

a) So they are equvalent cause by reads value in the context context. (reads state from .value by delegating, and using same mutable state.

b) In the text a off set is read in the ambda so it read in the layout phase no recomposition and for b its read in the body of the composable so its recomposed more work for the same result.

c) Sowhrn a state that compose read changes compose reexcute itself.so the reexcuition is the compose itself.

Snippets

```
// Snippet 1: reading through .value
val paddingState: MutableState<Dp> = remember { mutableStateOf(8.dp) }
Text(text = "Hello", modifier = Modifier.padding(paddingState.value))

// Snippet 2: reading through a property delegate
var padding: Dp by remember { mutableStateOf(8.dp) }
Text(text = "Hello", modifier = Modifier.padding(padding))

// Snippet 3: two ways to feed the same state into offset
var offsetX by remember { mutableStateOf(8.dp) }
Text("A", Modifier.offset { IntOffset(offsetX.roundToPx(), 0) })

val offsetPx = with(LocalDensity.current) { offsetX.roundToPx() }
Text("B", Modifier.offset { IntOffset(offsetPx, 0) })
```

Part 2 — Seeing the phases in Android Studio (Questions 4–8)

Question 4 — Make Compose talk: the first frame HANDS-ON — ANDROID STUDIO

Context: You cannot see phases, but you can make them print. Composition means that a composable function runs, so a `SideEffect` in its body prints once per composition. Layout and drawing can be observed from a modifier: the `layout` modifier reports measurement and placement, `drawBehind` reports drawing.

Task: Create the project, add the helper and build the lecture's tree. Run it and read Logcat with the filter `tag:Phases`. Write down the order of all log lines of the first frame and compare it with your answer to Question 2: does the measure order match the lecture's trace? Then rotate the emulator and say what runs again, and why.

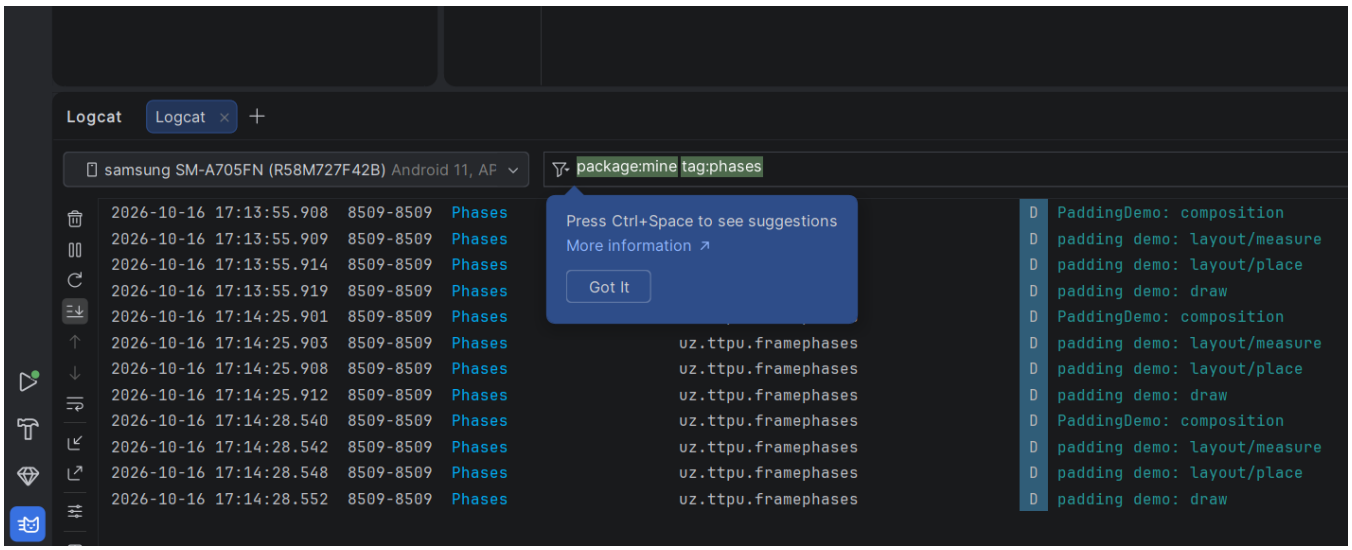
- New Project → Empty Activity (Compose): name `FramePhases`, package `uz.ttpu.framephases`, Minimum SDK API 24.

- Add `PhaseLog.kt` with the helper below, then show `LectureTree()` from `setContent`. The logs of this question are only for the experiment: a real app keeps side effects out of the body of a composable, in the effect APIs.

```
D Image: layout/place
D Column: layout/place
D Text A: layout/place
D Text B: layout/place
D Row: draw
D Image: draw
D Column: draw
D Text A: draw
D Text B: draw
D Text A: layout/measure
D Text B: layout/measure
D Text B: layout/place
D Text A: layout/place
D Row: draw
D Image: draw
D Column: draw
D Text A: draw
D Text B: draw
```

-

When ı totate



Whrn ı rotate the phone emulator works again ald reexcutes all the pahses. ı ll add a read me file to my new project for hands on questions answers.

PhaseLog.kt

```
/** Logs the layout phase (measure and placement) and the drawing phase
 *  of the element it is applied to. */
fun Modifier.logPhases(name: String): Modifier = this
    .layout { measurable, constraints ->
        Log.d("Phases", "$name: layout/measure")
        val placeable = measurable.measure(constraints)
        layout(placeable.width, placeable.height) {
            Log.d("Phases", "$name: layout/place")
            placeable.place(0, 0)
        }
    }
    .drawBehind { Log.d("Phases", "$name: draw") }
```

LectureTree.kt

```
@Composable
fun LectureTree() {
    SideEffect { Log.d("Phases", "LectureTree: composition") }

    Row(Modifier.logPhases("Row")) {
        Box(Modifier.size(40.dp).background(Color.Gray).logPhases("Image"))
        Column(Modifier.logPhases("Column")) {
            Text("Title text", Modifier.logPhases("Text A"))
            Text("Subtitle", Modifier.logPhases("Text B"))
        }
    }
}
```

Question 5 — Reading state in the composition phase HANDS-ON — ANDROID STUDIO

Context: A state read while a modifier chain is being built is read in the composition phase. This is the lecture's `Modifier.padding(padding)` example.

Task: Add `PaddingDemo` and show it. Before you tap, predict which of the lines `composition`, `layout/measure`, `layout/place` and `draw` will appear after one tap on Toggle padding. Then tap several times, compare with Logcat and write down any difference between your prediction and the log.

PaddingDemo.kt

```
@Composable
fun PaddingDemo() {
    var padding by remember { mutableStateOf(8.dp) }
    SideEffect { Log.d("Phases", "PaddingDemo: composition") }

    Column {
        Button(onClick = { padding = if (padding == 8.dp) 32.dp else 8.dp }) {
            Text("Toggle padding")
        }
        Text(
            text = "Padding demo",
            modifier = Modifier
                .padding(padding)               // where is `padding` read?
                .logPhases("padding demo"),
        )
    }
}
```

Question 6 — Reading state in the layout phase HANDS-ON — ANDROID STUDIO

Context: The lambda version `Modifier.offset { … }` runs during the placement step of the layout phase, so a state read inside that lambda is a layout-phase read. The lecture adds that the measurement step and the placement step have separate restart scopes.

Task: Add `OffsetDemo`, predict the log, tap Move and compare. Does a `composition` line appear? Does `layout/measure`? Then run the experiment below: move the read out of the lambda, tap again and say what changed in Logcat and why. Restore the original afterwards.

OffsetDemo.kt

```
@Composable
fun OffsetDemo() {
    var offsetX by remember { mutableStateOf(0.dp) }
    SideEffect { Log.d("Phases", "OffsetDemo: composition") }

    Column {
        Button(onClick = { offsetX = if (offsetX == 0.dp) 80.dp else 0.dp }) {
            Text("Move")
        }
        Text(
            text = "Offset demo",
            modifier = Modifier
                .offset { IntOffset(offsetX.roundToPx(), 0) }   // where is `offsetX` read?
                .logPhases("offset demo"),
```

```
        )
    }
}
```

Experiment: move the read above the Text

```
// new line, above the Text:
val offsetPx = with(LocalDensity.current) { offsetX.roundToPx() }

Text(
    text = "Offset demo",
    modifier = Modifier
        .offset { IntOffset(offsetPx, 0) }              // the lambda now uses the Int
        .logPhases("offset demo"),
)
```

Question 7 — Reading state in the drawing phase HANDS-ON — ANDROID STUDIO

Context: `Canvas`, `Modifier.drawBehind` and `Modifier.drawWithContent` read their state while drawing. The lecture says a change then runs only the drawing phase.

Task: Add `ColorDemo`, predict the log, tap and compare. Then replace the `Canvas` with `Box(Modifier.size(120.dp).background(color).logPhases("color demo"))` (the `background(Color)` overload receives the value when the modifier is built) and tap again. Which phases appear now? What does the difference tell you about where a state is read versus where it is stored? (The lecture: where a state is created has little bearing on the phases.)

`ColorDemo.kt`

```
@Composable
fun ColorDemo() {
    var color by remember { mutableStateOf(Color.Red) }
    SideEffect { Log.d("Phases", "ColorDemo: composition") }

    Column {
        Button(onClick = { color = if (color == Color.Red) Color.Blue else Color.Red }) {
            Text("Change color")
        }
        Canvas(Modifier.size(120.dp).logPhases("color demo")) {
            drawRect(color)                    // where is `color` read?
        }
    }
}
```

Question 8 — Which phase re-runs? THEORETICAL

Context: Compose notifies only the phases that have work to do for each affected element. Below are six places where a state is read.

Task: For each of (a)–(f) say in which phase the state is read, and which phases run when it changes: for composition, layout and drawing mark each as “runs”, “skipped” or “only if size or position changed”. Then choose which of the six you would use for a value that changes on every frame of an animation, and explain why.

I marked below so state is read on a on composition so all 3 runs for b layout , c composition , d drawing e layout cf drawing .

For position the changes in lambda (b) , for colors or visuals dor f (they skipp composition. A and c recompose everyfrom. Bad.

Six reads

```
(a)  Modifier.padding(p)                          // p read as the modifier is built  c : runs 1 :
runs :d runs
(b)  Modifier.offset { IntOffset(p, 0) }          // p read inside the lambda c:skipped
l:runs :d :only is size or position changes
(c)  Modifier.offset(p)                           // the Dp overload runs in all 3
(d)  Canvas(Modifier.size(50.dp)) { drawRect(c) } // c read inside onDraw skipped skipped runs
(e)  Layout(content) { m, cs -> /* uses s */ }    // s read in the measure lambda skipped runs if
p or s changes
(f)  Modifier.drawWithContent { /* uses c */ }    // c read inside the draw lambda skipped skipped
runs
```

Part 3 — Optimizing state reads (Questions 9–11)

Question 9 — Parallax: composition read versus layout read HANDS-ON — ANDROID STUDIO

Context: The lecture's optimization example offsets an element by half of a list's scroll offset. The first version reads `firstVisibleItemScrollOffset` while building the modifier, so on every scroll event the whole content is composed, measured, laid out and drawn again. The better version reads it inside `offset { }`.

Task: Build `ParallaxSlow`, scroll the list for about three seconds and note the last composition number in Logcat (`tag:Parallax`). Then copy it to `ParallaxFast`, replace only the `offset` with the lambda version below, change the log label to `fast`, scroll the same way and compare the numbers. State in one sentence which phases the slow version triggers on each scroll event, and which ones the fast version triggers. Optional cross-check: Tools → Layout Inspector on the running app, and look at the recomposition counts. (The effect itself is deliberately simple; the experiment is about the state read.)

ParallaxSlow.kt

```
@Composable
fun ParallaxSlow() {
    val listState = rememberLazyListState()
    val compositions = remember { intArrayOf(0) }
    SideEffect { Log.d("Parallax", "slow: composition #${++compositions[0]}") }

    Box {
        Box(
            Modifier
                .fillMaxWidth()
                .height(120.dp)
                .background(Color(0xFF6650A4))
                .offset(
                    y = with(LocalDensity.current) {
                        (listState.firstVisibleItemScrollOffset / 2).toDp()
                    }
                )
        )
        LazyColumn(state = listState) {
            items(100) { index ->
                Text("Item $index", Modifier.fillMaxWidth().padding(16.dp))
            }
        }
    }
}
```

ParallaxFast: replace the offset modifier with this

```
.offset {
    IntOffset(x = 0, y = listState.firstVisibleItemScrollOffset / 2)
}
```

Question 10 — derivedStateOf: fewer recompositions for a Boolean HANDS-ON — ANDROID STUDIO

Context: Sometimes a state has to be read in composition. Then you can still limit recompositions by filtering the changes. The lecture's tool is `derivedStateOf`, which converts one or more state objects into another state.

Task: Build `ScrollToTopNaive`: the Top button appears once the list is scrolled past the first item. Scroll slowly through about 30 items and note the last composition number (`tag:Derived`). Then switch to the `derivedStateOf` version below, repeat and compare. Why is the difference so large when the button's visibility changes only twice? Finally make the button work: use `rememberCoroutineScope` and `animateScrollToItem(0)` (replace TODO 3).

ScrollToTopNaive.kt

```
@Composable
fun ScrollToTopNaive() {
    val listState = rememberLazyListState()
    val compositions = remember { intArrayOf(0) }
    SideEffect { Log.d("Derived", "naive: composition #${++compositions[0]}") }

    val showButton = listState.firstVisibleItemIndex > 0

    Box(Modifier.fillMaxSize()) {
        LazyColumn(state = listState) {
            items(200) { index ->
                Text("Item $index", Modifier.fillMaxWidth().padding(16.dp))
            }
        }
        if (showButton) {
            Button(
                onClick = { /* TODO 3 */ },
                modifier = Modifier.align(Alignment.BottomEnd).padding(16.dp),
            ) {
                Text("Top")
            }
        }
    }
}
```

The derivedStateOf version: replace the `val showButton = …` line with this

```
val showButton by remember { derivedStateOf { listState.firstVisibleItemIndex > 0 } }
```

Question 11 — Skipping and deferred reads HANDS-ON — ANDROID STUDIO

Context: Compose skips a composable function whose inputs did not change. Which functions run again after a state change depends on where the state is read.

Task: (1) Add `Parent`, `Child` and `Sibling`. Predict, then tap Add three times and count the `composition` lines for each function (`tag:Skip`). (2) Make the change below, so that `Child` receives a lambda instead of a value. Predict again, tap three times and compare. (3) Explain in two sentences why `Parent` no longer recomposes, using the term restart scope.

Parent.kt

```
@Composable
fun Parent() {
    var count by remember { mutableIntStateOf(0) }
    SideEffect { Log.d("Skip", "Parent: composition") }

    Column {
        Button(onClick = { count++ }) { Text("Add") }
        Child(count)
        Sibling()
    }
}

@Composable
fun Child(count: Int) {
    SideEffect { Log.d("Skip", "Child: composition") }
```

```
    Text("Count: $count")
}

@Composable
fun Sibling() {
    SideEffect { Log.d("Skip", "Sibling: composition") }
    Text("I never change")
}
```

The change: pass the value as a lambda

```
// in Parent:
Child(count = { count })              // pass a lambda that returns the value

// the Child:
@Composable
fun Child(count: () -> Int) {
    SideEffect { Log.d("Skip", "Child: composition") }
    Text("Count: ${count()}")         // the state is read here, not in Parent
}
```

Part 4 — Cyclic phase dependency (Questions 12–14)

Question 12 — Two frames for one layout THEORETICAL

Context: The phases never run backwards within one frame, but an app can still get into a loop across frames. In the lecture's example, `onSizeChanged` stores the image's measured height in a state, and the text below is padded by that state.

Task: Assume the image is 360 px tall. Describe frame 1 and frame 2: the value of `imageHeightPx` that the composition reads, the top padding of the text, what `onSizeChanged` does during the layout phase, whether a recomposition is scheduled, and what the text looks like when the frame is drawn. Then answer: (a) why is the `toDp()` conversion already a warning sign? (b) name the general three-step pattern to be careful of; (c) which principle from the lecture does this code break?

F1 : Composition reads imageHeightPx = 0.

The top padding of the text is 0 dp.

During the layout phase the image is measured as 360 px tall, and onSizeChanged runs and sets imageHeightPx = 360.

Because a state that was read in composition changed, a recomposition is scheduled.

When frame 1 is drawn, the text sits at the top, on top of the image (wrong picture).

F2: omposition reads imageHeightPx = 360.

The top padding of the text is 360 px converted to dp.

In layout the image size is still 360 px, so onSizeChanged sees no change and no new recomposition is scheduled.

The frame is drawn with the text below the image (correct picture).

A) The `toDp()` conversion is a warning sign because it turns a measured pixel size (information that only exists after the layout phase) into a value used in composition. That means the code is feeding layout information back into composition.

b) The pattern to be careful about: read a state in composition, then change that same state in a later phase (layout or drawing), which triggers another recomposition. (Composition → Layout → state write → Composition again.)

c) It breaks the principle of unidirectional data flow of the phases: information should flow in one direction (Composition → Layout → Drawing) and never backwards within a frame. The lecture's fix is to use a layout primitive such as Column, or a custom Layout, so the parent that measures the image also places the text.

So it supoorts phases never run backwards within one frame.

The lecture's example

```
Box {
    var imageHeightPx by remember { mutableIntStateOf(0) }

    Image(
        painter = painterResource(R.drawable.rectangle),
        contentDescription = "I'm above the text",
        modifier = Modifier
            .fillMaxWidth()
            .onSizeChanged { size ->
                imageHeightPx = size.height          // Don't do this
            }
    )

    Text(
        text = "I'm below the image",
        modifier = Modifier.padding(
            top = with(LocalDensity.current) { imageHeightPx.toDp() }
        )
    )
}
```

Question 13 — Reproduce the loop and fix it with a layout primitive HANDS-ON — ANDROID STUDIO

Context: A frame that needs a second frame to become correct wastes work and shows an intermediate picture to the user.

Task: Add `CyclicDemo` (a purple box stands for the image) and read the log (`tag:Cycle`): how many compositions happen, and with which values of `imageHeightPx`? Then add `CyclicFixed`, the same screen built with a `Column`, and compare the logs. Finally explain which node plays the role of the single source of truth in the fixed version.

CyclicDemo.kt

```
@Composable
fun CyclicDemo() {
    var imageHeightPx by remember { mutableIntStateOf(0) }
    val heightInThisComposition = imageHeightPx
    val compositions = remember { intArrayOf(0) }
    SideEffect {
        Log.d("Cycle", "composition #${++compositions[0]}: " +
            "imageHeightPx=$heightInThisComposition")
    }

    Box {
        Box(
            Modifier
                .fillMaxWidth()
                .height(120.dp)
                .background(Color(0xFF6650A4))
                .onSizeChanged { size ->
                    Log.d("Cycle", "onSizeChanged: height=${size.height}")
                    imageHeightPx = size.height          // Don't do this
                }
        )
        Text(
            text = "I'm below the image",
            modifier = Modifier.padding(
                top = with(LocalDensity.current) { heightInThisComposition.toDp() }
            ),
        )
    }
}
```

CyclicFixed.kt

```
@Composable
fun CyclicFixed() {
    val compositions = remember { intArrayOf(0) }
    SideEffect { Log.d("Cycle", "fixed: composition #${++compositions[0]}") }

    Column {
        Box(Modifier.fillMaxWidth().height(120.dp).background(Color(0xFF6650A4)))
        Text("I'm below the image")
    }
}
```

Question 14 — A custom layout when no primitive fits HANDS-ON — ANDROID STUDIO

Context: The lecture says a more complex version of the same problem may need a custom layout. Writing one is the three-step algorithm in code: measure the children, decide your own size, place the children.

Task: Complete `StackBelow` (three TODOs) and use it as shown to replace `CyclicDemo`: the screen must look like `CyclicFixed`, with an 8.dp gap between the image and the text, and no state at all. Read `tag:StackBelow` in Logcat: how many `measure child` lines appear before each `place children` line, and what does that show about the "one pass" rule? Then add a third child to the call and check how the count changes.

StackBelow.kt (write the TODOs)

```
@Composable
fun StackBelow(
    modifier: Modifier = Modifier,
    spacing: Dp = 8.dp,
    content: @Composable () -> Unit,
) {
    Layout(content = content, modifier = modifier) { measurables, constraints ->

        // Step 1: measure the children, each exactly once,
        // with loose minimum constraints.
        val loose = constraints.copy(minWidth = 0, minHeight = 0)
        val placeables = measurables.map { measurable ->
            // TODO 1: log "measure child", then measure it with `loose`
        }

        // Step 2: decide our own size: the widest child, and the heights
        // plus the gaps between them.
        val spacingPx = spacing.roundToPx()
        val width = 0    // TODO 2
        val height = 0   // TODO 2

        // Step 3: place the children, one below the other.
        layout(constraints.constrainWidth(width), constraints.constrainHeight(height)) {
            // TODO 3: log "place children", then place every child
            //         below the previous one
        }
    }
}
```

Usage

```
StackBelow(Modifier.fillMaxWidth(), spacing = 8.dp) {
    Box(Modifier.fillMaxWidth().height(120.dp).background(Color(0xFF6650A4)))
    Text("I'm below the image")
}
```